

BDA501 – Big Data

Lecture 8: Real-Time Data Processing with Kafka and Spark Streaming

Cao Vu Bui

FSB

March 2026

Lecture 8 Overview

Topics for today:

- Batch vs real-time data processing
- Introduction to data streaming
- Apache Kafka architecture
- Kafka topics and producers
- Spark Structured Streaming
- Streaming data pipelines
- Real-time analytics use cases

Recap from Lecture 7

Previously we studied:

- Spark MLlib
- feature engineering
- distributed machine learning
- scalable ML pipelines

However, these pipelines often rely on **batch data processing**.
Modern systems increasingly require **real-time analytics**.

Batch vs Real-Time Processing

Batch Processing:

- processes large datasets periodically
- high throughput
- high latency

Real-Time Processing:

- processes data continuously
- low latency
- near real-time insights

Examples include:

- financial transactions
- IoT sensors
- social media streams

What is Data Streaming?

Data streaming refers to:

continuous processing of data as it arrives.

Characteristics:

- unbounded data streams
- event-driven systems
- real-time processing

Streaming platforms enable scalable event processing.

Apache Kafka

Apache Kafka is a distributed event streaming platform.

Key features:

- high-throughput messaging system
- distributed architecture
- fault tolerance
- scalable event streaming

Kafka is widely used in modern data pipelines.

Kafka Architecture



Kafka Topics and Partitions

A Kafka topic stores streams of messages.

Topics are divided into:

- partitions
- distributed across brokers

Benefits:

- parallel processing
- high scalability
- fault tolerance

Kafka Producer and Consumer

Producer:

- sends messages to Kafka topics
- publishes event streams

Consumer:

- reads messages from topics
- processes streaming data

Multiple consumers can process data in parallel.

Spark Structured Streaming

Spark Structured Streaming processes real-time data streams.

Key idea:

Streaming queries are treated as incremental batch queries.

Advantages:

- unified batch and streaming API
- fault tolerance
- scalable distributed processing

Reading Streaming Data

Example: reading from Kafka

```
df = spark.readStream  
    .format("kafka")  
    .option("subscribe", "sensor-data")  
    .load()
```

Spark continuously reads streaming data.

Streaming Processing

Example transformation:

```
result = df.groupBy("device_id") \  
            .count()
```

Spark processes streaming data incrementally.

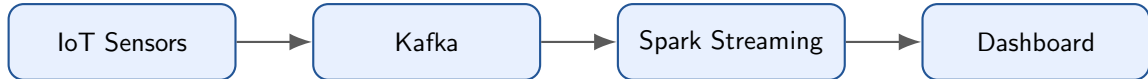
Writing Streaming Output

Example output:

```
query = result.writeStream \  
    .outputMode("complete") \  
    .format("console") \  
    .start()
```

The stream runs continuously.

Example Streaming Pipeline



Real-Time Analytics Use Cases

Common streaming applications:

- fraud detection
- recommendation systems
- IoT monitoring
- online analytics dashboards
- predictive maintenance

Real-time systems enable immediate decision making.

Key Takeaways

- real-time processing is essential for modern systems
- Kafka enables scalable event streaming
- Spark Structured Streaming processes real-time data
- streaming pipelines support low-latency analytics
- streaming architectures power modern data platforms